

SDD Pilot – Discussion Guide for PM & Engineering

Purpose: Facilitate discussion about implementing Spec Driven Development (SDD) as a pilot with your cross-functional team.

Participants: Product Manager, Product Designer, Engineering Lead

Date: 2026-03-24

What is Spec Driven Development (SDD)?

A structured approach where **Epics** (in ProductBoard/Jira) define the business case (WHAT & WHY), and **Spec files** (in Git) provide detailed implementation blueprints (HOW).

The Problem We're Solving:

- Requirements are incomplete or change during development
- Handoffs between PM/Design/Engineering are messy
- Information scattered across ProductBoard, Jira, Confluence, Figma, Mural, Dovetail
- Team members get overwhelmed or miss critical details
- Knowledge is lost when people transition off projects
- AI coding assistants lack structured context

The Solution:

- **Epic** = High-level business case (stays in ProductBoard/Jira)
 - **Spec** = Detailed implementation blueprint (lives in Git with code)
 - Clear boundary between business planning and technical execution
 - Single source of truth for implementation details
-

Epic vs Spec – Quick Comparison

Aspect	Epic (ProductBoard/Jira)	Spec (Git Repository)
Owner	Product Manager	Collaborative (PM + Design + Eng)
Purpose	Business planning, prioritization, tracking	Implementation blueprint, AI-friendly context
Focus	WHAT & WHY	HOW
Audience	Stakeholders, leadership, roadmap planning	PM, Design, Engineering, AI assistants
Location	ProductBoard/Jira	Git repo (version-controlled with code)
Contains	User problem, business value, success metrics, high-level scope, team assignments, timeline	Detailed user flows, technical architecture, data models, API contracts, testing strategy

Aspect	Epic (ProductBoard/Jira)	Spec (Git Repository)
Detail Level	High-level ("Users can export data")	Specific ("Export button top-right, CSV/JSON, max 10k rows")
Updates	Throughout feature lifecycle	Locked at approval, updated if scope changes
Visibility	Business, product, engineering	Engineering, design, technical implementation
Examples	"Enable data export to increase retention by 15%"	"POST /api/exports, sync <1000 rows, async >1000 rows, S3 storage, email notification"

The Epic vs Spec Boundary

Epic (ProductBoard/Jira)

Owner: Product Manager **Purpose:** Business planning, prioritization, tracking

Contains:

- User problem & business opportunity
- Success metrics (KPIs, OKRs)
- Links to research (Dovetail), journey maps (Mural)
- High-level scope ("Users can export their data")
- Team assignments, dependencies, timeline
- Business value & priority

Spec File (Git Repository)

Owner: Collaborative (PM drafts context, Design adds UX, Engineering adds technical) **Purpose:** Implementation blueprint, AI-friendly context

Contains:

- Link back to Epic for context
- Detailed user flows & design decisions (consolidates from Figma)
- Technical architecture & component breakdown
- Data models & API contracts
- Detailed acceptance criteria ("Export button top-right, CSV/JSON, max 10k rows...")
- Edge cases & error handling
- Testing strategy

Example Relationships:

- **One Epic → One Spec:** Most common (straightforward features)
- **One Epic → Multiple Specs:** Complex work broken into phases
- **Multiple Epics → One Spec:** Rare (shared technical foundation)

The Workflow

1. PM creates Epic in ProductBoard/Jira
↓
2. PM creates Spec file (fills Context & Background from Epic)
Status: Draft
↓
3. Designer fills User Experience section
Status: Still Draft
↓
4. Engineering fills Technical Approach & Testing Strategy
Status: Still Draft
↓
5. Team reviews together, resolves Open Questions, all sign off
Status: In Review → Approved
↓
6. Engineering implements (references Spec during development)
Epic status: In Progress
↓
7. Feature complete
Epic status: Done
Spec status: Implemented

Golden Rule: No coding starts until Spec status = "Approved"

What's In The Framework

We've created:

1. **Design Document** (<docs/superpowers/specs/2026-03-24-spec-driven-development-design.md>)
 - Complete framework design with Epic/Spec boundary, workflow, and pilot plan
 2. **Team README** (<specs/README.md>)
 - Quick-start guide: when to create specs, workflow, tips
 3. **Spec Template** (<specs/TEMPLATE.md>)
 - Ready-to-use template with all sections and ownership annotations
 4. **Implementation Plan** (<docs/superpowers/plans/2026-03-24-sdd-framework-enhancements.md>)
 - Plan to add status guide, example spec, and AI integration guide
 5. **Example Spec** (<specs/EXAMPLE-2026-03-csv-export.md>)
 - Reference example showing a complete, realistic spec
-

Discussion Questions

1. Epic/Spec Boundary Clarity

- Does the WHAT/WHY (Epic) vs HOW (Spec) boundary make sense?
- Are there fields in our current Epics that should move to Specs?
- Are there fields that should stay in both (with Epic as source of truth)?

2. Workflow Fit

- Does this workflow match how we actually collaborate?
- Where might this create friction or slow us down?
- What steps could we streamline for our team?

3. Spec Granularity

- **One Epic → One Spec:** Does this feel right for most of our features?
- **One Epic → Multiple Specs:** When would we break work into multiple specs?
- How do we decide when to create a spec vs. just update the Epic?

Sizing Exercise: Think about a recent feature. Would it be:

- One spec (straightforward, 1-3 sprints)?
- Multiple specs (complex, needs phasing)?
- Too small for a spec (bug fix, minor change)?

4. Pilot Feature Selection

Ideal pilot feature characteristics:

- Medium complexity (not too simple, not overwhelming)
- Involves all three roles (PM, Design, Engineering)
- Has clear design AND technical components
- ~1-2 sprints duration
- NOT on a critical deadline (room to experiment)

Candidates from our backlog:

- [Discuss features that meet these criteria]

Selected pilot feature: _____

5. Ownership & Responsibilities

Who owns each Spec section?

- Context & Background → PM (clear?)
- User Experience → Design (clear?)
- Technical Approach → Engineering (clear?)
- Acceptance Criteria → Shared (how do we collaborate on this?)

Who reviews & approves?

- All three roles must sign off before "Approved" status

- How will we handle disagreements during review?

6. Process Concerns

Potential challenges:

- "This feels like more documentation work" → How do we keep specs lean?
- "We already have Epics with lots of detail" → What moves from Epic to Spec?
- "What if requirements change during implementation?" → How do we keep Epic & Spec in sync?

Mitigation strategies:

- Link to Epic instead of duplicating content
- Focus on details needed for implementation (not documentation for its own sake)
- Update Spec inline for small changes, team discussion for big changes

7. Success Metrics for Pilot

How will we know if SDD worked?

Primary Goal: Team finds the process valuable enough to continue

Secondary Indicators:

- Reduced back-and-forth during implementation ("wait, I thought we were doing X")
- Fewer Slack questions about requirements (team references Spec instead)
- Better alignment before implementation starts (all roles clear on approach)
- Spec becomes go-to reference during development

What we'll measure (observe during pilot):

- Spec completion rate (did all sections get filled?)
- Time to approval (was review process smooth?)
- Implementation confidence (did Eng feel clear on what to build?)
- Reference frequency (did team actually use Spec during dev?)

8. Timeline

Before Pilot Feature:

- Review this framework (this meeting)
- Select pilot feature (this meeting or follow-up)
- Brief full team on workflow (15-min overview)

During Pilot Feature:

- PM creates Epic → PM creates Spec → Design contributes → Eng contributes → Team reviews → Implementation
- Track: How often did we reference the Spec? How many clarifying questions were avoided?

After Pilot Feature:

- Retrospective (30-60 min)

- What worked well?
 - What felt like overhead?
 - Would we use this again?
 - Decide: Expand, adjust and re-pilot, or shelve
-

Next Steps (From This Meeting)

Decisions Needed:

- ☐ Does the Epic/Spec boundary make sense to us?
- ☐ Does the workflow fit our team's collaboration style?
- ☐ What's our preferred spec granularity (sizing)?
- ☐ Which feature should be the pilot?
- ☐ Who will facilitate the team briefing?
- ☐ When do we want to start the pilot?

Action Items:

- ☐ **PM:** _____
- ☐ **Design:** _____
- ☐ **Engineering:** _____

Follow-Up Meeting (if needed): _____

Resources

- **Full Design Doc:** <docs/superpowers/specs/2026-03-24-spec-driven-development-design.md>
 - **Team README:** <specs/README.md>
 - **Spec Template:** <specs/TEMPLATE.md>
 - **Example Spec:** <specs/EXAMPLE-2026-03-csv-export.md>
-

Open Questions / Concerns

Use this space to capture questions or concerns that come up during discussion:

1. _____
 2. _____
 3. _____
-

Remember: This is a pilot. We're testing to see if this process works for us. We can adjust, iterate, or abandon based on what we learn. The goal is better collaboration and clearer implementation plans - if the process doesn't serve that goal, we change it.

Navigation

Getting Started: [Team README](#) · [Spec Template](#) · [Example Spec](#)

Guides: [Status Tracking](#) · [AI Integration](#) · [Pilot Discussion](#)

Framework: [Design Document](#) · [CLAUDE.md](#)